

**TRƯỜNG ĐẠI HỌC LẠC HỒNG
KHOA CÔNG NGHỆ THÔNG TIN**



LAC HONG UNIVERSITY
Powered by Arizona State University®

BÁO CÁO ĐỒ ÁN MÔN HỌC

ĐỀ TÀI:

**HỆ THỐNG GIÁM SÁT VÀ DỰ ĐOÁN
CHẤT LƯỢNG KHÔNG KHÍ SỬ DỤNG EDGE
AI
(AIR QUALITY AIoT)**

GVHD: Hồ Ngọc Hoàng Long

Lớp: 23CT111

SVTH: Tống Anh Kha - 123000367

Nguyễn Thanh Nam - 123000156

Đinh Thị Quỳnh Anh - 123000379

Nguyễn Ngô Tôn Thái - 123000558

Đồng Nai - 2026

Tóm tắt đề án

Đề án trình bày quá trình thiết kế, lập trình và triển khai hoàn chỉnh một hệ thống AIoT (Artificial Intelligence of Things) có khả năng giám sát chất lượng không khí đa chỉ số và dự báo nồng độ bụi mịn PM2.5 trong thời gian thực trực tiếp trên vi điều khiển ESP32-S3.

Kiến trúc phần cứng bao gồm vi điều khiển ESP32-S3-WROOM-1 N16R8 (16 MB Flash, 8 MB PSRAM) làm nhân xử lý trung tâm, phối hợp với ba cảm biến môi trường: BME280 (nhiệt độ, độ ẩm, áp suất khí quyển qua giao tiếp I2C), Sharp GP2Y1014AU0F (nồng độ bụi PM2.5 qua ADC) và MQ135 (khí độc hại và VOCs qua ADC), màn hình IPS ST7789 240×240 px giao tiếp SPI, cùng các cơ cấu cảnh báo vật lý gồm còi buzzer và đèn LED đỏ.

Đóng góp kỹ thuật nổi bật gồm ba tầng xử lý tích hợp liền mạch: (1) Bộ lọc dữ liệu thường toán học Z-Score kết hợp Đường trung bình động lũy thừa (EMA, $\alpha = 0,2$) loại bỏ các gai nhiễu đột biến từ cảm biến quang học trước khi đưa vào mô hình; (2) Mô hình mạng nơ-ron hồi quy LSTM (Long Short-Term Memory) được huấn luyện trên Python và nén bằng kỹ thuật lượng tử hóa sau huấn luyện INT8 (Post-Training Quantization), nhúng thành công vào phân vùng PSRAM với thời gian suy luận thực tế đo được là **216 ms** mỗi chu kỳ; (3) Cơ chế đồng bộ đám mây hai tầng lên Firebase Realtime Database theo chu kỳ 5 giây (dữ liệu tức thời) và 60 giây (nhật ký lịch sử).

Giao diện Web Dashboard được triển khai tại <https://aiot-air-quality.web.app> theo phong cách Glassmorphism tối giản, hiển thị đồng thời ba luồng dữ liệu (PM2.5 thô, PM2.5 đã lọc nhiễu và PM2.5 dự báo AI), hỗ trợ tải trước 30 bản ghi lịch sử nhằm đảm bảo biểu đồ liên tục ngay từ thời điểm truy cập. Kết quả thực nghiệm xác nhận hệ thống phát hiện và chặn thành công xung nhiễu PM2.5 lên đến $506,6 \mu\text{g}/\text{m}^3$ và tự động kích hoạt cảnh báo âm thanh khi mô hình AI dự báo ô nhiễm vượt ngưỡng $80 \mu\text{g}/\text{m}^3$.

Từ khóa: AIoT, Edge AI, ESP32-S3, TensorFlow Lite Micro, LSTM, Z-Score, PM2.5, Firebase, Glassmorphism.

Mục lục

1	Giới thiệu chung	5
1.1	Đặt vấn đề	5
1.2	Mục tiêu của đồ án	5
1.3	Phạm vi và đối tượng nghiên cứu	6
2	Thiết kế Hệ thống và Sơ đồ Phần cứng	7
2.1	Kiến trúc tổng thể hệ thống	7
2.2	Danh mục thiết bị sử dụng (Bill of Materials)	7
2.3	Mạch lọc thông thấp RC bảo vệ cảm biến bụi Sharp	8
2.4	Sơ đồ đấu nối chi tiết (Pinout Mapping)	9
2.5	Khuyến nghị cấp nguồn hệ thống	10
2.6	Sơ đồ cấu trúc phần cứng (Block Diagram)	10
3	Cơ sở Lý thuyết và Thuật toán	12
3.1	Chuẩn hóa dữ liệu đầu vào (Feature Scaling – MinMaxScaler)	12
3.2	Thuật toán Phát hiện dị thường (Anomaly Detection – Z-Score + EMA)	13
3.2.1	Động lực và bài toán đặt ra	13
3.2.2	Nguyên lý hoạt động bộ lọc Z-Score động kết hợp EMA	13
3.2.3	Hiện thực hóa trong firmware C++	14
3.3	Mạng nơ-ron Hồi quy LSTM (Long Short-Term Memory)	15
3.3.1	Lý do lựa chọn LSTM cho bài toán dự báo chuỗi thời gian	15
3.3.2	Cấu trúc toán học của một ô nhớ LSTM	15
3.3.3	Cấu trúc mô hình LSTM nhúng trong hệ thống	16
3.4	Lượng tử hóa mô hình (Post-Training Quantization INT8)	17
3.4.1	Sự cần thiết của lượng tử hóa	17
3.4.2	Nguyên lý lượng tử hóa INT8 tuyến tính đối xứng	17
3.4.3	Hiệu quả đạt được	18
3.5	Ý nghĩa của mô hình Edge AI trong hệ thống	18
4	Triển khai Phần mềm trên Thiết bị (Firmware)	19
4.1	Cấu trúc dự án và thư viện phụ thuộc	19
4.2	Luồng hoạt động chính của chương trình	19
4.2.1	Hàm setup() – Khởi tạo một lần	19
4.2.2	Hàm loop() – Vòng lặp chính (chu kỳ 5 giây)	20
4.3	Sơ đồ hoạt động phần mềm (Flowchart)	21
4.4	Cấu hình bộ nhớ PSRAM và khởi tạo mô hình TensorFlow Lite Micro	21
4.5	Kỹ thuật Double-buffering trên màn hình LCD ST7789	22
4.6	Quản lý WiFi động với WiFiManager và cơ chế khôi phục cấu hình	23

4.7	Đồng bộ dữ liệu đa tầng lên Firebase Realtime Database	24
5	Giao diện Giám sát trực tuyến (Web Dashboard)	26
5.1	Kiến trúc và công nghệ Web Dashboard	26
5.2	Thiết kế giao diện Glassmorphism	26
5.3	Thuật toán tải trước lịch sử (History Pre-loading)	26
5.4	Sơ đồ giao tiếp bất đồng bộ (UML Sequence Diagram)	28
5.5	Tối ưu hóa hiển thị trên thiết bị di động (Responsive Design)	28
5.6	Triển khai lên Firebase Hosting	28
6	Kết quả Thực nghiệm và Đánh giá	30
6.1	Đo đặc hiệu năng suy luận AI trên vi điều khiển	30
6.2	Thử nghiệm bộ lọc dị thường Z-Score – Kịch bản gai nhiều nhân tạo	30
6.3	Logic kích hoạt cơ cấu cảnh báo theo dự báo AI	31
6.4	Hình ảnh sản phẩm thực tế	32
6.5	Đánh giá tổng thể và điểm hạn chế	32
7	Kết luận và Hướng phát triển	34
7.1	Tóm tắt kết quả đạt được	34
7.2	Hướng phát triển tương lai	34

1 Giới thiệu chung

1.1 Đặt vấn đề

Trong bối cảnh đô thị hóa và công nghiệp hóa diễn ra với tốc độ nhanh chóng, ô nhiễm không khí đang trở thành một trong những mối đe dọa sức khỏe cộng đồng nghiêm trọng nhất tại Việt Nam và trên toàn thế giới. Đặc biệt, các hạt bụi mịn PM_{2.5} – được định nghĩa là các hạt lơ lửng có đường kính khí động học nhỏ hơn hoặc bằng 2,5 micromet – có khả năng xuyên qua hệ thống lọc của mũi và họng người, đi sâu vào phế nang phổi và thậm chí thâm nhập vào mạch máu. Phơi nhiễm dài hạn với PM_{2.5} được Tổ chức Y tế Thế giới (WHO) ghi nhận là nguyên nhân trực tiếp gây ra các bệnh về hô hấp mãn tính (viêm phế quản, hen suyễn), bệnh tim mạch và ung thư phổi [1]. Theo hướng dẫn chất lượng không khí toàn cầu của WHO năm 2021, nồng độ PM_{2.5} trung bình hàng năm không được vượt quá $5 \mu\text{g}/\text{m}^3$, trong khi phần lớn các thành phố lớn tại Việt Nam thường xuyên có chỉ số vượt mức cho phép nhiều lần.

Hiện nay, hầu hết các trạm quan trắc chất lượng không khí trong nước đều hoạt động theo mô hình *thụ động*: chúng chỉ đo đạc và hiển thị giá trị tức thời của các chỉ số môi trường mà không có khả năng phân tích xu hướng hoặc đưa ra cảnh báo sớm về các đợt ô nhiễm sắp xảy ra. Điều này đặt người dùng vào thế bị động, chỉ biết phản ứng sau khi đã tiếp xúc với không khí ô nhiễm.

Ngoài ra, mô hình xử lý dữ liệu tập trung trên đám mây (Cloud Computing) – nơi dữ liệu thô từ cảm biến được truyền toàn bộ lên máy chủ để phân tích – đối mặt với nhiều thách thức trong thực tế triển khai tại Việt Nam: độ trễ truyền tin cao (đặc biệt tại các khu vực có kết nối Internet không ổn định), tiêu tốn băng thông, chi phí máy chủ và nguy cơ mất toàn bộ tính năng cảnh báo khi mất kết nối mạng. Hơn nữa, việc truyền liên tục dữ liệu lên đám mây cũng tiềm ẩn rủi ro về bảo mật và quyền riêng tư dữ liệu.

Mặt khác, các cảm biến bụi giá rẻ loại quang học (optical dust sensor) vốn rất nhạy cảm với các tác nhân gây nhiễu cục bộ tạm thời như: khói thuốc lá, bụi đất do quét nhà, dị vật bay ngang qua vùng cảm nhận, hoặc ngay cả hơi nước ngưng tụ. Những gai nhiễu đột biến này nếu không được lọc bỏ trước khi đưa vào mô hình học máy sẽ khiến các dự báo bị sai lệch nghiêm trọng, làm mất đi ý nghĩa của hệ thống cảnh báo.

Do đó, đồ án này đề xuất một giải pháp tích hợp toàn diện: xây dựng thiết bị AIoT nhúng trực tiếp thuật toán lọc nhiễu và mô hình trí tuệ nhân tạo ngay trên vi điều khiển tại biên (Edge AI), đảm bảo thiết bị vẫn hoạt động và cảnh báo tự động ngay cả khi hoàn toàn mất kết nối Internet.

1.2 Mục tiêu của đồ án

Mục tiêu cốt lõi của đồ án bao gồm bốn điểm chính:

1. **Thiết kế phần cứng trạm đo đa chỉ số:** Chế tạo thiết bị đo đồng thời năm chỉ số môi trường gồm nồng độ bụi PM_{2.5} (sử dụng cảm biến Sharp GP2Y1014), nồng độ khí gas

và VOCs (cảm biến MQ135), nhiệt độ, độ ẩm tương đối và áp suất khí quyển (cảm biến BME280), đồng thời hiển thị kết quả trực tiếp trên màn hình LCD IPS ST7789 tại chỗ.

2. **Triển khai thuật toán lọc nhiễu Z-Score+EMA trên vi điều khiển:** Lập trình bộ phát hiện dị thường bằng ngôn ngữ C++ trực tiếp trên ESP32-S3, sử dụng thuật toán Z-Score động kết hợp Đường trung bình động lũy thừa (EMA) để phát hiện và triệt tiêu các gai nhiễu đột biến cục bộ từ cảm biến quang học, đảm bảo đầu vào sạch cho mô hình AI.
3. **Nhúng mô hình LSTM lượng tử hóa INT8 chạy tại biên:** Huấn luyện mô hình mạng nơ-ron LSTM trên Python, áp dụng kỹ thuật Post-Training Quantization (INT8) để nén mô hình, rồi tích hợp thành công lên phân vùng PSRAM của ESP32-S3 thông qua thư viện TensorFlow Lite Micro, đạt thời gian suy luận thực tế dưới 250 ms cho mỗi chu kỳ dự báo.
4. **Xây dựng hệ sinh thái Cloud và Web Dashboard:** Thiết lập pipeline đồng bộ dữ liệu lên Firebase Realtime Database và xây dựng giao diện Web Dashboard theo phong cách Glassmorphism, cung cấp khả năng giám sát từ xa đa thiết bị (PC và điện thoại di động) thông qua đường dẫn truy cập công khai bảo mật HTTPS.

1.3 Phạm vi và đối tượng nghiên cứu

Đề án tập trung nghiên cứu và triển khai trên một thiết bị đơn lẻ, phù hợp để đặt trong môi trường *indoor* (phòng ngủ, văn phòng) hoặc *bán-outdoor* (ban công, hành lang). Đối tượng nghiên cứu chính là chuỗi thời gian (time-series) của các chỉ số môi trường được thu thập với tần suất 5 giây/mẫu. Mô hình LSTM sử dụng cửa sổ trượt 24 bước thời gian liên tiếp trong quá khứ (tương đương 2 phút dữ liệu tích lũy trong điều kiện cảm biến đo liên tục) để dự báo nồng độ PM2.5 trong 1 giờ tiếp theo.

Giới hạn phạm vi: Đề án không đề cập đến việc xây dựng mạng lưới nhiều trạm quan trắc phân tán, không thiết kế ứng dụng di động native, và chưa tích hợp module năng lượng pin sạc (hệ thống hiện tại sử dụng nguồn USB/Adapter cố định).

2 Thiết kế Hệ thống và Sơ đồ Phần cứng

2.1 Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo mô hình ba tầng kết nối liền mạch với nhau:

1. **Tầng cảm biến và xử lý biên (Edge Layer):** Vi điều khiển ESP32-S3-WROOM-1 N16R8 là trái tim của hệ thống, điều phối thu thập dữ liệu từ ba cảm biến, thực thi thuật toán lọc nhiễu Z-Score/EMA và chạy suy luận AI bằng TensorFlow Lite Micro, rồi điều khiển các cơ cấu cảnh báo và hiển thị lên màn hình ST7789.
2. **Tầng đám mây (Cloud Layer):** Firebase Realtime Database của Google đóng vai trò trung gian lưu trữ và phân phối dữ liệu theo thời gian thực. Dữ liệu được ESP32 đẩy lên theo hai nhánh: /sensor & /ai (cập nhật tức thời mỗi 5 giây) và /history (nhập ký tích lũy mỗi 60 giây).
3. **Tầng giao diện người dùng (Presentation Layer):** Web Dashboard được viết bằng HTML/CSS/Vanilla JavaScript và phát hành lên Firebase Hosting, cho phép truy cập giám sát từ xa trên mọi thiết bị có trình duyệt web.

2.2 Danh mục thiết bị sử dụng (Bill of Materials)

Việc lựa chọn linh kiện được cân nhắc kỹ lưỡng để đảm bảo thiết bị đáp ứng đồng thời ba tiêu chí: năng lực tính toán đủ mạnh để chạy mô hình AI, độ chính xác cảm biến đủ cao cho ứng dụng giám sát môi trường, và tổng chi phí hợp lý cho một đồ án môn học. Danh mục linh kiện đầy đủ được trình bày trong Bảng 1.

Bảng 1: Danh mục linh kiện chính trong hệ thống (Bill of Materials)

Linh kiện	Thông số kỹ thuật	Vai trò trong hệ thống
ESP32-S3-WROOM-1 (N16R8)	Dual-core Xtensa LX7 240 MHz; 16 MB Flash (QIO); 8 MB PSRAM (OPI)	Vi điều khiển trung tâm: thu thập dữ liệu, lọc nhiễu, chạy mô hình AI, điều khiển ngoại vi và truyền dữ liệu qua WiFi
Sharp GP2Y1014AU0F	Cảm biến bụi quang học hồng ngoại; ngõ ra Analog; điện áp hoạt động 5 V DC	Đo nồng độ hạt bụi PM2.5 sơ bộ (chỉ số tương đối), cần mạch lọc RC bảo vệ
MQ135 Gas Sensor & Cảm biến điện trở hóa học (bán dẫn); nhạy với NH ₃ , NO _x , CO ₂ , Benzen; ngõ ra Analog; tiêu thụ dòng ≈150 mA (sấy nóng) & Đo điện áp tương đối của khí độc hại và VOCs, dùng để cảnh báo nguy cơ cháy nổ và ô nhiễm khí		
ST7789 IPS LCD	1,54 inch; độ phân giải 240 × 240 px; giao tiếp SPI; tấm nền IPS góc nhìn rộng	Hiển thị Dashboard tại chỗ: chỉ số PM2.5, dự báo AI, nhiệt độ, độ ẩm, trạng thái WiFi và cảnh báo dị thường
Còi Buzzer chủ động	Còi active 5 V (KY-012); tần số cộng hưởng ≈2,3 kHz	Phát âm thanh cảnh báo theo xung PWM khi AI dự báo ô nhiễm vượt ngưỡng
LED đỏ 5 mm	LED thông thường, $V_f \approx 2,1$ V; kết hợp điện trở hạn dòng 220 Ω	Đèn báo trạng thái ô nhiễm nghiêm trọng
MB102 Power Module	Cấp nguồn 5 V và 3,3 V; dòng tối đa 700 mA; kết nối cổng DC Barrel	Cấp nguồn ngoài ổn định, cách ly để tránh sụt áp khi MQ135 sấy nhiệt và cảm biến bụi nháy xung LED

2.3 Mạch lọc thông thấp RC bảo vệ cảm biến bụi Sharp

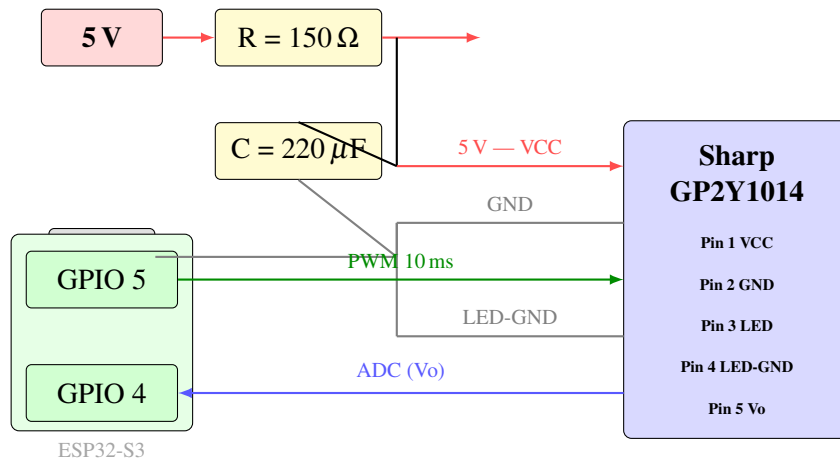
Cảm biến bụi quang học Sharp GP2Y1014AU0F hoạt động theo nguyên lý: vi điều khiển tạo một xung điều khiển 10 ms lên chân LED (chân số 3), đèn LED hồng ngoại bên trong cảm biến bật sáng và chiếu vào buồng đo. Ánh sáng bị tán xạ bởi hạt bụi tạo ra tín hiệu điện áp tương tự ở chân Vo (chân số 5), vi điều khiển đọc tín hiệu này qua ADC để suy ra nồng độ bụi.

Tuy nhiên, theo datasheet của Sharp, bóng LED hồng ngoại bên trong cảm biến yêu cầu dòng đỉnh khá lớn và ổn định trong mỗi xung sáng. Nếu cấp nguồn 5 V trực tiếp không qua lọc, mỗi khi LED bật sẽ gây ra một gai điện áp tức thời trên đường nguồn, ảnh hưởng đến điện áp tham chiếu ADC của ESP32 và gây ra giá trị đo sai. Mạch lọc RC (Bộ lọc thông thấp RC) bắt buộc phải được trang bị với các tham số:

$$\tau = R \cdot C = 150\,\Omega \times 220\,\mu\text{F} = 33\,\text{ms}$$

(1)

Hằng số thời gian $\tau = 33 \text{ ms}$ đủ lớn để lọc phẳng các xung nhiễu cao tần trên đường nguồn ($f_{\text{nhiều}} > 1/\tau \approx 30 \text{ Hz}$), trong khi vẫn đảm bảo LED đáp ứng đủ nhanh với chu kỳ điều khiển 10 ms theo yêu cầu datasheet. Sơ đồ đấu dây mạch lọc RC:



Hình 1: Block Diagram: Mạch lọc RC ($R = 150 \Omega$, $C = 220 \mu\text{F}$, $\tau = 33 \text{ ms}$) bảo vệ cảm biến Sharp GP2Y1014 – kết nối ESP32-S3

2.4 Sơ đồ đấu nối chi tiết (Pinout Mapping)

Bảng phân bổ chân GPIO của ESP32-S3 được thiết kế để tránh xung đột phần cứng giữa các giao thức và tận dụng tối đa băng thông bus. Cụ thể, các thiết bị SPI được gán vào các chân SPI Hardware (MOSI = GPIO 11, SCLK = GPIO 12) để hỗ trợ DMA tốc độ cao, trong khi I2C sử dụng GPIO 8/9 theo cấu hình mặc định của thư viện Wire trên ESP32-S3. Chi tiết phân bổ chân được trình bày trong Bảng 2.

Bảng 2: Bảng phân bổ chân GPIO của ESP32-S3-WROOM-1 N16R8

Thiết bị ngoại vi	Tín hiệu	GPIO (ESP32-S3)	Giao thức / Chế độ
Cảm biến BME280	SDA	GPIO 8	I2C Master – Wire()
	SCL	GPIO 9	I2C Master – Wire()
Cảm biến Sharp GP2Y1014	Vo (Analog Out)	GPIO 4	ADC1 – Đọc điện áp
	LED (Control)	GPIO 5	Digital Output – Xung 10 ms
Cảm biến MQ135	AO (Analog Out)	GPIO 1	ADC1 – Đọc điện áp
Màn hình ST7789 LCD	SDA (MOSI)	GPIO 11	SPI Hardware MOSI
	SCL (SCLK)	GPIO 12	SPI Hardware Clock
	CS (Chip Select)	GPIO 10	Digital Output
	DC (Data/Command)	GPIO 13	Digital Output
	RST (Reset)	GPIO 14	Digital Output
	BLK (Backlight)	GPIO 15	PWM – Điều chỉnh độ sáng
Còi Buzzer	Tín hiệu	GPIO 6	Digital Output / PWM tone()
LED cảnh báo đỏ	Dương cực (+)	GPIO 7	Digital Output
Nút BOOT (reset WiFi)	Chân ấn	GPIO 0	Digital Input Pull-up

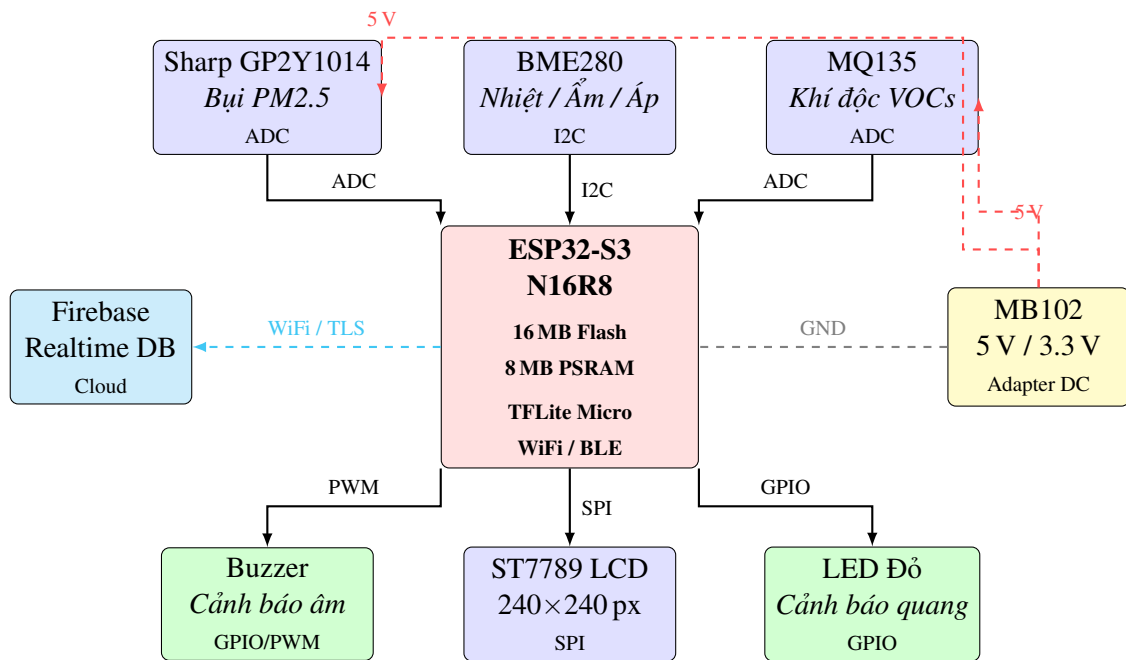
2.5 Khuyến nghị cấp nguồn hệ thống

Đây là điểm thiết kế phần cứng cực kỳ quan trọng, thường bị bỏ qua khi thiết kế sơ bộ: cảm biến khí MQ135 có bộ phận sấy nóng điện trở bên trong tiêu thụ dòng điện lên tới 150 mA liên tục. Đồng thời, bóng LED hồng ngoại của cảm biến bụi Sharp nháy xung dòng cao tần số liên tiếp. Nếu cả hai thiết bị này lấy nguồn 5 V trực tiếp từ đường VBUS của cổng USB của ESP32 (thường giới hạn 500 mA), gộp với dòng tiêu thụ của chính ESP32 ($\approx 200\text{--}300$ mA khi WiFi hoạt động), hệ thống sẽ rất dễ rơi vào trạng thái sụt áp (Brown-out), gây ra hiện tượng ESP32 tự khởi động lại ngẫu nhiên.

Giải pháp bắt buộc là sử dụng Module nguồn MB102 cấp điện áp 5 V độc lập từ Adapter cắm tường cho MQ135 và mạch lọc RC của cảm biến Sharp, đồng thời nối chung mặt phẳng GND giữa nguồn ngoài và ESP32-S3 để bảo toàn tham chiếu điện áp ADC.

2.6 Sơ đồ cấu trúc phần cứng (Block Diagram)

Hình 2 thể hiện toàn bộ mối quan hệ vật lý, giao thức truyền thông và đường cấp nguồn giữa ESP32-S3 và các khối chức năng của hệ thống theo dạng Block Diagram chuẩn kỹ thuật.



Hình 2: Block Diagram: Kiến trúc kết nối phần cứng và giao thức truyền thông của hệ thống

3 Cơ sở Lý thuyết và Thuật toán

3.1 Chuẩn hóa dữ liệu đầu vào (Feature Scaling – MinMaxScaler)

Các mô hình mạng nơ-ron học sâu, đặc biệt là LSTM, rất nhạy cảm với biên độ của dữ liệu đầu vào. Nếu các đặc trưng có biên độ khác nhau quá lớn (ví dụ: áp suất ≈ 1013 hPa trong khi $PM_{2.5} \approx 30 \mu g/m^3$), gradient trong quá trình huấn luyện sẽ bị thống trị bởi các đặc trưng có giá trị lớn, dẫn đến hội tụ chậm hoặc không hội tụ. Do đó, tất cả dữ liệu đầu vào phải được chuẩn hóa về dải $[0, 1]$ thống nhất bằng thuật toán MinMaxScaler tuyến tính:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (2)$$

Các cặp hằng số $[x_{\min}, x_{\max}]$ được xác định cố định dựa trên dải vật lý tối đa thực tế của mỗi đặc trưng và phải **hoàn toàn đồng bộ** giữa môi trường huấn luyện Python và firmware C++ nhúng trên ESP32. Bảng 3 trình bày chi tiết các hằng số này, khớp chính xác với mảng `FEATURE_MIN[]` và `FEATURE_MAX[]` được khai báo trong `main.cpp`.

Bảng 3: Hằng số MinMaxScaler đồng bộ giữa Python và C++ Firmware

Đặc trưng (Feature)	Chỉ số (Index)	$x_{\min}$	$x_{\max}$	Đơn vị
PM2.5 (nồng độ bụi mịn)	0	0,0	1000,0	$\mu g/m^3$
Độ ẩm / Điểm sương	1	-40,0	30,0	% RH
Nhiệt độ	2	-20,0	45,0	$^{\circ}C$
Áp suất khí quyển	3	990,0	1050,0	hPa

Sau khi mô hình LSTM thực hiện suy luận và trả về kết quả dự báo dưới dạng chuẩn hóa $y_{\text{scaled}} \in [0, 1]$, hệ thống thực hiện phép biến đổi ngược (Inverse MinMaxScaler) để phục hồi giá trị vật lý thực tế:

$$y_{\text{raw}} = y_{\text{scaled}} \times (y_{\max} - y_{\min}) + y_{\min} \quad (3)$$

Trong firmware C++, hai phép biến đổi này được hiện thực hóa bằng hàm `scaleData()` và `inverseScale()` trong `main.cpp`. Hàm `scaleData()` còn tích hợp thêm cơ chế kẹp giá trị (clamping) để xử lý các trường hợp NaN từ cảm biến lỗi:

```

1 const float FEATURE_MIN[4] = {0.0f, -40.0f, -20.0f, 990.0f};
2 const float FEATURE_MAX[4] = {1000.0f, 30.0f, 45.0f, 1050.0f};
3
4 // Linear normalization: raw value --> [0, 1]
5 float scaleData(float value, int feature_index) {
6     if (isnan(value))
7         value = FEATURE_MIN[feature_index]; // Default to MIN if NaN

```

```

8      // Clamp to valid range
9      value = max(value, FEATURE_MIN[feature_index]);
10     value = min(value, FEATURE_MAX[feature_index]);
11     return (value - FEATURE_MIN[feature_index]) /
12            (FEATURE_MAX[feature_index] - FEATURE_MIN[feature_index]);
13 }
14
15 // Inverse transform: [0, 1] --> actual physical value
16 float inverseScale(float scaled_value, int feature_index) {
17     return scaled_value *
18            (FEATURE_MAX[feature_index] - FEATURE_MIN[feature_index]) +
19            FEATURE_MIN[feature_index];
20 }

```

Listing 1: MinMaxScaler normalization and inverse scale functions in firmware C++

3.2 Thuật toán Phát hiện dị thường (Anomaly Detection – Z-Score + EMA)

3.2.1 Động lực và bài toán đặt ra

Cảm biến bụi quang học Sharp GP2Y1014 hoạt động bằng cách đếm số lượng hạt bụi tán xạ ánh sáng hồng ngoại trong buồng đo. Cơ chế này khiến cảm biến cực kỳ nhạy cảm với bất kỳ tác nhân nào tạo ra mật độ hạt cao đột ngột trong không gian xung quanh: khói thuốc lá thổi thẳng vào cảm biến, bụi đất từ hoạt động quét nhà gần kề, con kiến hay hạt bụi lớn bay ngang qua vùng cảm nhận, hoặc thậm chí hơi nước ngưng tụ. Các tác nhân này tạo ra **gai nhiễu đột biến (Anomaly Spike)** – giá trị đọc vọt lên cực cao tức thời (ví dụ: từ 30 lên 500⁺ $\mu\text{g}/\text{m}^3$) rồi trở về bình thường ngay lập tức.

Nếu giá trị gai nhiễu này được đưa trực tiếp vào bộ đệm trượt của LSTM, mô hình sẽ nhận nhầm đây là ô nhiễm thực sự và đưa ra dự báo sai lệch nghiêm trọng trong nhiều chu kỳ tiếp theo (do cơ chế bộ nhớ Cell State của LSTM). Do đó, bộ lọc dị thường là tầng bảo vệ không thể thiếu.

3.2.2 Nguyên lý hoạt động bộ lọc Z-Score động kết hợp EMA

Hệ thống sử dụng **Đường trung bình động lũy thừa (Exponential Moving Average – EMA)** để xây dựng một đường cơ sở thích nghi (adaptive baseline) liên tục cập nhật theo thời gian:

$$\text{EMA}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \text{EMA}_{t-1} \quad (4)$$

Hệ số làm mượt $\alpha = 0,2$ (được khai báo trong firmware là $\text{EMA_ALPHA} = 0.2\text{f}$) được chọn để cân bằng giữa khả năng phản ứng nhanh với xu hướng dài hạn thực sự và khả năng kháng nhiễu ngắn hạn. Phương sai di động (Moving Variance) tương ứng được cập nhật song song:

$$\text{Var}_t = \alpha \cdot (x_t - \text{EMA}_{t-1})^2 + (1 - \alpha) \cdot \text{Var}_{t-1} \quad (5)$$

Độ lệch chuẩn di động: $\sigma_t = \sqrt{\text{Var}_t}$. Để tránh trường hợp chia cho không khi môi trường ổn định ($\sigma_t \approx 0$), firmware đặt sàn tối thiểu $\sigma_{\min} = 10,0 \mu\text{g}/\text{m}^3$:

$$\sigma_t^{\text{eff}} = \max(\sigma_t, \sigma_{\min}) = \max(\sqrt{\text{Var}_t}, 10,0) \quad (6)$$

Điểm Z-Score tuyệt đối của mẫu đo mới x_t so với đường cơ sở:

$$Z_t = \frac{|x_t - \text{EMA}_{t-1}|}{\sigma_t^{\text{eff}}} \quad (7)$$

Quy tắc phán định:

- Nếu $Z_t \leq 3,0$: Mẫu x_t được xác định là dữ liệu **bình thường**. Cho phép đi qua và cập nhật EMA, Var theo công thức (4) và (5).
- Nếu $Z_t > 3,0$: Mẫu x_t được xác định là **gai nhiễu dị thường** (xác suất sai $< 0,3\%$ theo phân phối chuẩn). Giá trị thô bị **chặn lại** và thay thế bằng EMA_{t-1} trước khi đưa vào bộ đệm LSTM. EMA và Var **không được cập nhật** để duy trì đường cơ sở ổn định.

Ngưỡng $Z > 3,0$ tương ứng với quy tắc 3σ của phân phối chuẩn Gauss, bao phủ 99,7% dữ liệu hợp lệ trong vùng chấp nhận được.

3.2.3 Hiện thực hóa trong firmware C++

```

1 float ema_pm25 = -1.0f;    // EMA baseline (uninitialized)
2 float ema_variance = 0.0f; // Moving variance (initial value)
3 const float EMA_ALPHA = 0.2f; // Smoothing factor
4 const float ANOMALY_THRESHOLD_Z = 3.0f; // 3-sigma threshold
5
6 bool isAnomaly(float current_pm25) {
7     // First reading: initialize baseline
8     if (ema_pm25 < 0) {
9         ema_pm25 = current_pm25;
10        return false;
11    }
12
13    float diff = current_pm25 - ema_pm25;
14    float stddev = sqrt(ema_variance);
15    if (stddev < 10.0f) stddev = 10.0f; // Minimum floor to prevent division
16                                     // by zero

```

```

17     float z_score = abs(diff) / stddev;
18
19     if (z_score > ANOMALY_THRESHOLD_Z) {
20         return true; // Anomaly spike detected!
21     }
22
23     // Update EMA and variance only for clean (non-anomaly) readings
24     ema_pm25 = (EMA_ALPHA * current_pm25) + ((1.0f - EMA_ALPHA) * ema_pm25);
25     ema_variance = (EMA_ALPHA * diff * diff) + ((1.0f - EMA_ALPHA) *
26         ema_variance);
27     return false;
28 }

```

Listing 2: Z-Score + EMA Anomaly Detection filter in main.cpp

3.3 Mạng nơ-ron Hồi quy LSTM (Long Short-Term Memory)

3.3.1 Lý do lựa chọn LSTM cho bài toán dự báo chuỗi thời gian

Dự báo nồng độ ô nhiễm không khí là bài toán dự báo chuỗi thời gian điển hình: giá trị PM2.5 tại thời điểm tương lai phụ thuộc vào lịch sử các giá trị trong quá khứ (mô hình thống kê không gian-thời gian). Các mạng nơ-ron truyền thẳng thông thường (Feedforward Neural Networks) không có cơ chế ghi nhớ, không thể khai thác thông tin phụ thuộc thời gian này. Mạng RNN (Recurrent Neural Network) cơ bản có cơ chế hồi quy nhưng bị giới hạn bởi vấn đề *tiêu biến đạo hàm (Vanishing Gradient)*: gradient bị thu nhỏ theo cấp số nhân qua nhiều bước thời gian, khiến mạng không thể học được các phụ thuộc dài hạn quan trọng (ví dụ: xu hướng tích lũy bụi trong 6 giờ trước đó).

LSTM, được Hochreiter và Schmidhuber đề xuất năm 1997 [2], giải quyết vấn đề này thông qua cơ chế **Đường dẫn tế bào nhớ (Cell State)** – một đường truyền gradient thẳng xuyên suốt theo thời gian – kết hợp với ba cổng kiểm soát thông minh quyết định thông tin nào cần ghi nhớ, quên đi và xuất ra.

3.3.2 Cấu trúc toán học của một ô nhớ LSTM

Cho đầu vào tại bước thời gian t là x_t , trạng thái ẩn từ bước trước là h_{t-1} và trạng thái ô nhớ từ bước trước là C_{t-1} , các phương trình hoạt động của một ô LSTM được định nghĩa như sau:

$$\text{Cổng Quên (Forget Gate): } f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (8)$$

$$\text{Cổng Vào (Input Gate): } i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (9)$$

$$\text{Trạng thái ô nhớ tạm: } \tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (10)$$

$$\text{Cập nhật ô nhớ: } C_t = f_t \circ C_{t-1} + i_t \circ \tilde{C}_t \quad (11)$$

$$\text{Cổng Ra (Output Gate): } o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (12)$$

$$\text{Trạng thái ẩn đầu ra: } h_t = o_t \circ \tanh(C_t) \quad (13)$$

Trong đó: σ là hàm sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$; $\circ$ biểu thị phép nhân phần tử (Hadamard product); W_f, W_i, W_c, W_o là các ma trận trọng số và b_f, b_i, b_c, b_o là các vector độ lệch (bias) tương ứng với bốn cổng.

Trực quan hóa vai trò các cổng: Cổng Quên f_t (giá trị gần 0 – quên, gần 1 – giữ) quyết định thông tin dài hạn nào trong Cell State cũ C_{t-1} cần bị xóa đi. Cổng Vào i_t và trạng thái tạm $\tilde{C}_t$ phối hợp xác định thông tin mới từ đầu vào hiện tại x_t sẽ được ghi vào ô nhớ với mức độ bao nhiêu. Cổng Ra o_t quyết định phần nào của ô nhớ hiện tại C_t sẽ được xuất ra thành trạng thái ẩn h_t truyền cho bước tiếp theo.

3.3.3 Cấu trúc mô hình LSTM nhúng trong hệ thống

Mô hình AI được nhúng vào ESP32-S3 có kiến trúc tối giản nhưng hiệu quả, phù hợp với tài nguyên phần cứng hạn chế:

- **Input Shape:** [1, 24, 4] – batch size 1, chuỗi 24 bước thời gian liên tiếp, mỗi bước có 4 đặc trưng (PM2.5, Độ ẩm/Điểm sương, Nhiệt độ, Áp suất).
- **LSTM Layer:** 32 units ẩn (hidden neurons). Các ma trận trọng số cổng W_f, W_i, W_c, W_o có kích thước $32 \times (32 + 4) = 32 \times 36$, tương đương 1152 trọng số cho mỗi cổng, tổng cộng $4 \times 1152 = 4608$ trọng số và $4 \times 32 = 128$ biases chỉ riêng tầng LSTM.
- **Dense Output Layer:** 1 nơ-ron, thực hiện phép biến đổi tuyến tính:

$$y_{\text{scaled}} = W_d \cdot h_{24} + b_d, \quad W_d \in \mathbb{R}^{1 \times 32}, b_d \in \mathbb{R} \quad (14)$$

Đầu ra $y_{\text{scaled}} \in [0, 1]$ là giá trị PM2.5 đã chuẩn hóa, cần áp dụng Inverse MinMaxScaler (3) để phục hồi về $\mu\text{g}/\text{m}^3$.

Bộ đệm vòng (Circular Buffer) trong firmware lưu trữ $24 \times 4 = 96$ giá trị lượng tử hóa INT8, được khai báo là `int8_t input_buffer[24][4]`. Mỗi chu kỳ 5 giây, toàn bộ bộ đệm dịch trái 1 bước (mô phỏng cửa sổ trượt), mẫu mới nhất được ghi vào hàng cuối cùng `input_buffer[23]`.

3.4 Lượng tử hóa mô hình (Post-Training Quantization INT8)

3.4.1 Sự cần thiết của lượng tử hóa

Vi điều khiển ESP32-S3, dù là một trong những vi điều khiển mạnh nhất trên thị trường hiện tại, vẫn có những giới hạn tính toán và bộ nhớ căn bản so với máy chủ GPU: không có FPU vector (SIMD) để tính toán số thực dấu phẩy động 32-bit hàng loạt, RAM tích hợp chỉ có 320 KB. Một mô hình LSTM Float32 thông thường có kích thước 4.8 KB chỉ cho ma trận trọng số, nhưng khi hoạt động trong TFLite cần thêm bộ nhớ đệm trung gian (activation tensors), tổng Tensor Arena yêu cầu lên tới vài MB. Do đó, lượng tử hóa là bước bắt buộc để mô hình chạy được tại biên.

3.4.2 Nguyên lý lượng tử hóa INT8 tuyến tính đối xứng

Lượng tử hóa chuyển đổi mỗi trọng số số thực $r \in \mathbb{R}$ (Float32, 4 byte) thành số nguyên 8-bit $q \in [-128, 127]$ (INT8, 1 byte) thông qua phép ánh xạ tuyến tính có tham số:

$$q = \text{clamp}\left(\text{round}\left(\frac{r}{S}\right) + Z, -128, 127\right) \quad (15)$$

Trong đó: $S > 0$ là hệ số tỷ lệ (Scale factor) và Z là điểm không tuyệt đối (Zero-point offset), cả hai được xác định trên mỗi layer từ dữ liệu hiệu chỉnh (calibration dataset). Phép biến đổi ngược tại thời điểm suy luận:

$$r \approx S \cdot (q - Z) \quad (16)$$

Trong firmware, sau khi interpreter->Invoke() hoàn tất, giá trị đầu ra INT8 cần được giải lượng tử hóa thông qua các tham số scale và zero_point của tensor đầu ra:

```

1 // Read predicted INT8 value from output tensor
2 int8_t predicted_val = output->data.int8[0];
3
4 // Dequantize: INT8 --> [0, 1] (float32)
5 float dequantized_val = (predicted_val - output->params.zero_point)
6                       * output->params.scale;
7
8 // Inverse MinMaxScaler: [0,1] --> actual ug/m3
9 float final_pm25 = inverseScale(dequantized_val, 0);
10 // final_pm25 = predicted PM2.5 concentration for next 1 hour (ug/m3)

```

Listing 3: Dequantize LSTM output and restore actual PM2.5 value

3.4.3 Hiệu quả đạt được

Việc lượng tử hóa toàn bộ mô hình về INT8 mang lại ba lợi ích đồng thời:

1. **Giảm bộ nhớ 4 lần:** Mỗi trọng số Float32 (4 byte) được rút gọn xuống INT8 (1 byte). File .tflite từ vài MB giảm xuống còn ≈ 40 KB cho mảng C-array nhúng trực tiếp vào Flash.
2. **Tăng tốc tính toán:** ESP32-S3 thực hiện phép nhân ma trận số nguyên nhanh hơn đáng kể so với số thực. Thời gian suy luận thực tế chỉ 216 ms, chiếm chưa đến 5% chu kỳ 5 giây của vòng lặp chính.
3. **Giảm tiêu thụ năng lượng:** Phép tính số nguyên tiêu tốn ít điện năng hơn phép tính số thực dấu phẩy động.

3.5 Ý nghĩa của mô hình Edge AI trong hệ thống

Điện toán biên (Edge AI) là yếu tố tạo nên sự khác biệt căn bản của hệ thống này so với các thiết bị IoT truyền thống chỉ thu thập và truyền dữ liệu lên đám mây. Khi mô hình LSTM chạy trực tiếp trên ESP32-S3:

- **Hoạt động độc lập khi mất mạng:** Thiết bị tiếp tục phát hiện dị thường, chạy suy luận AI và kích hoạt còi/đèn cảnh báo ngay cả khi WiFi hoàn toàn mất kết nối. Người dùng tại chỗ vẫn được bảo vệ.
- **Độ trễ cực thấp:** Suy luận hoàn tất trong 216 ms ngay tại thiết bị, không có độ trễ mạng 100–500 ms như khi gửi lên đám mây.
- **Bảo mật dữ liệu:** Dữ liệu cảm biến được xử lý cục bộ, chỉ kết quả tổng hợp (trạng thái hiện tại và dự báo) mới được gửi lên đám mây.
- **Giảm chi phí đám mây:** Chỉ gửi tối đa một gói dữ liệu nhỏ gọn mỗi 5 giây thay vì luồng dữ liệu thô liên tục.

4 Triển khai Phần mềm trên Thiết bị (Firmware)

4.1 Cấu trúc dự án và thư viện phụ thuộc

Dự án firmware được tổ chức theo cấu trúc module hóa rõ ràng bằng công cụ PlatformIO, cho phép biên dịch và nạp code hiệu quả:

```

1 esp32-air-quality/
2 +-- platformio.ini      # Board config, libraries, build flags
3 +-- model/              # model_data.cpp (C-array INT8)
4 +-- src/
5     +-- main.cpp        # Main loop: AI inference, anomaly detection
6     +-- ai/
7         | +-- model_data.h # TFLite model array declaration
8     +-- sensors/
9         | +-- sensor_manager.cpp/h # BME280, GP2Y1014, MQ135 drivers
10    +-- display/
11        | +-- display_manager.cpp/h # ST7789 + Sprite double-buffering
12    +-- network/
13        +-- wifi_manager.cpp/h # WiFiManager + BOOT button reset
14        +-- firebase_manager.cpp/h # Firebase Realtime DB sync

```

Listing 4: Cấu trúc thư mục dự án firmware (PlatformIO)

Danh sách thư viện bên ngoài (khai báo trong `platformio.ini`):

- Adafruit BME280 Library ■2.2.4: Driver đọc cảm biến BME280 qua I2C.
- Firebase Arduino Client Library ■4.4.14: SDK kết nối Firebase Realtime Database.
- TensorFlowLite_ESP32 ■1.0.0: Thư viện TensorFlow Lite Micro cho ESP32.
- TFT_eSPI ■2.5.43: Driver màn hình ST7789, hỗ trợ SPI DMA và Sprite buffer.
- WiFiManager (tzapu): Cấu hình WiFi động qua Access Point.

4.2 Luồng hoạt động chính của chương trình

4.2.1 Hàm `setup()` – Khởi tạo một lần

Khi thiết bị cấp nguồn hoặc reset, hàm `setup()` thực hiện theo trình tự:

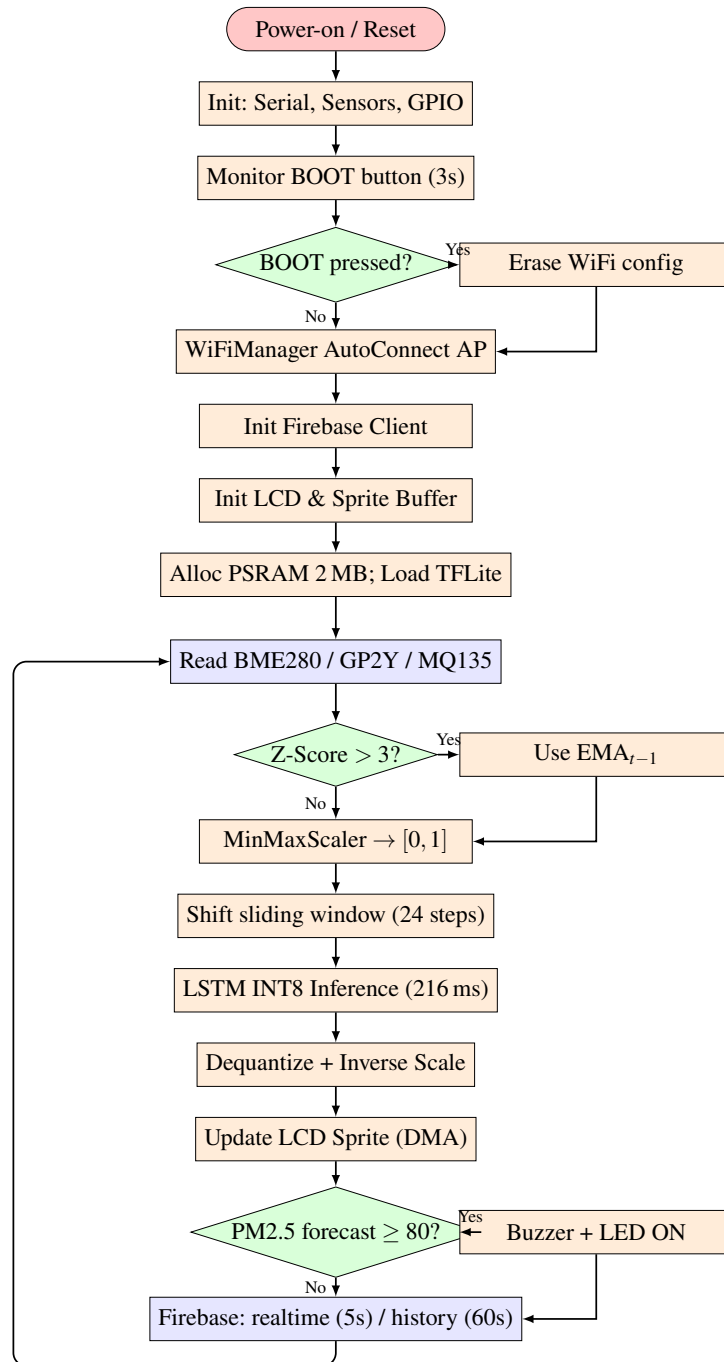
1. Khởi tạo cổng Serial (115200 baud) để in log debug.
2. Gọi `sensorManager.begin()` để khởi tạo bus I2C, cấu hình GPIO cho cảm biến Sharp và MQ135.

3. Cấu hình GPIO 6 (Buzzer) và GPIO 7 (LED) là Digital Output và đặt mặc định LOW.
4. Gọi `wifiManager.init()` – kiểm tra nút BOOT trong 3 giây; nếu bị nhấn, xóa cấu hình WiFi cũ và phát Access Point `AirQuality_AIoT_Setup`.
5. Gọi `firebaseManager.init()` để xác thực với Firebase.
6. Gọi `displayManager.begin()` để khởi tạo ST7789 và cấp phát Sprite buffer trong PSRAM.
7. Cấp phát Tensor Arena 2 MB trong PSRAM qua `heap_caps_malloc(MALLOC_CAP_SPIRAM)`.
8. Nạp mô hình TFLite từ mảng `model_data[]`, khởi tạo Resolver, Interpreter và gọi `AllocateTensors`.

4.2.2 Hàm `loop()` – Vòng lặp chính (chu kỳ 5 giây)

1. Đọc dữ liệu thô từ ba cảm biến thông qua `sensorManager.readAll()`.
2. Kiểm tra gai nhiễu PM2.5 bằng `isAnomaly()`; nếu phát hiện, thay x_t bằng EMA_{t-1} và đặt cờ `anomaly_detected = true`.
3. Dịch bộ đệm trượt sang trái 1 bước (23 bước cũ) và ghi mẫu mới vào `input_buffer[23]`.
4. Chuẩn hóa 4 đặc trưng bằng `scaleData()` rồi lượng tử hóa INT8 thông qua tham số `tensor.scale` và `.zero_point`.
5. Gọi `interpreter->Invoke()` để thực hiện suy luận AI (LSTM INT8).
6. Giải lượng tử hóa kết quả và áp dụng `Inverse MinMaxScaler` thu được `final_pm25`.
7. Cập nhật màn hình LCD thông qua `displayManager.updateData()`.
8. Kiểm tra ngưỡng cảnh báo: nếu `final_pm25 >= 80.0  $\mu\text{g}/\text{m}^3$` , kích hoạt còi `buzzer tone(PIN_BUZZER, 1000, 1000)` và bật LED GPIO 7.
9. Đồng bộ dữ liệu lên Firebase theo chu kỳ 5 giây (realtime) và 60 giây (history log).

4.3 Sơ đồ hoạt động phần mềm (Flowchart)



Hình 3: Flowchart: Luồng hoạt động phần mềm từ khởi động đến vòng lặp chính

4.4 Cấu hình bộ nhớ PSRAM và khởi tạo mô hình TensorFlow Lite Micro

RAM nội bộ của ESP32-S3 chỉ có 320 KB, không đủ để lưu Tensor Arena cho mô hình LSTM cùng với các activation buffer trung gian. Hệ thống bắt buộc phải cấp phát động Tensor Arena trên bộ nhớ ngoài PSRAM 8 MB thông qua hàm `heap_caps_malloc()` với cờ `MALLOC_CAP_SPIRAM`:

```

1 #include <TensorFlowLite_ESP32.h>
2 #include "tensorflow/lite/micro/all_ops_resolver.h"
3 #include "tensorflow/lite/micro/micro_interpreter.h"
4
5 constexpr int kTensorArenaSize = 2 * 1024 * 1024; // 2 MB in PSRAM
6 uint8_t* tensor_arena = nullptr;
7
8 // Inside setup():
9 tensor_arena = (uint8_t*)heap_caps_malloc(kTensorArenaSize, MALLOC_CAP_SPIRAM);
10 if (!tensor_arena) {
11     // Fallback: try allocating in internal RAM (may fail for large size)
12     tensor_arena = (uint8_t*)malloc(kTensorArenaSize);
13 }
14
15 // Load model from C-array embedded in Flash
16 model = tflite::GetModel(model_data);
17
18 // Initialize Resolver (register all required ops)
19 static tflite::AllOpsResolver resolver;
20 static tflite::MicroErrorReporter error_reporter;
21
22 // Initialize Interpreter with Arena allocated in PSRAM
23 static tflite::MicroInterpreter static_interpreter(
24     model, resolver, tensor_arena, kTensorArenaSize, &error_reporter);
25 interpreter = &static_interpreter;
26
27 // Allocate memory for all input/output/intermediate tensors
28 interpreter->AllocateTensors();
29
30 // Get pointers to input and output tensors
31 input = interpreter->input(0); // Shape: [1, 24, 4], dtype: INT8
32 output = interpreter->output(0); // Shape: [1, 1], dtype: INT8

```

Listing 5: Allocate Tensor Arena in PSRAM and initialize TFLite Interpreter

4.5 Kỹ thuật Double-buffering trên màn hình LCD ST7789

Giao diện màn hình cần cập nhật liên tục mỗi 5 giây với nhiều thành phần đồ họa: tiêu đề, giá trị PM2.5 lớn, thanh chỉ báo, dự báo AI, nhiệt độ, độ ẩm và biểu tượng trạng thái WiFi. Nếu vẽ trực tiếp từng phần lên màn hình theo thứ tự, người dùng sẽ thấy màn hình cập nhật từng phần

một, gây hiệu ứng *chớp* (*flickering*) rõ ràng và làm giảm tính chuyên nghiệp của giao diện.

Giải pháp **Double-buffering** sử dụng đối tượng TFT_eSprite của thư viện TFT_eSPI: toàn bộ khung hình mới được vẽ vào một vùng nhớ ảo (Sprite) trong PSRAM trước, sau đó đẩy toàn bộ Sprite lên màn hình vật lý trong một thao tác DMA duy nhất cực nhanh (không thấy được bởi mắt người):

```

1 void DisplayManager::updateData(float temp, float hum, float raw_pm25,
2   float filtered_pm25, float predicted_pm25, float gas_volts,
3   bool wifi_connected, bool anomaly) {
4
5   // === PHASE 1: Draw entire new frame into Sprite (in PSRAM) ===
6   sprite.fillRect(COLOR_BG); // Clear sprite with background color
7   drawHeader(wifi_connected); // Draw header + WiFi icon
8   drawPM25Card(filtered_pm25, raw_pm25, anomaly); // PM2.5 gauge card
9   drawAIPredictionCard(predicted_pm25); // AI forecast card
10  drawEnvironmentStats(temp, hum, gas_volts); // Temp, Hum, Gas
11
12  // === PHASE 2: Push entire Sprite from PSRAM to LCD at once (DMA) ===
13  // This transfer is so fast that no flickering is visible to human eye
14  sprite.pushSprite(0, 0);
15 }
```

Listing 6: Flicker-free LCD update using Sprite Double-buffering technique

4.6 Quản lý WiFi động với WiFiManager và cơ chế khôi phục cấu hình

Để thiết bị có thể di chuyển giữa các môi trường mạng khác nhau (nhà riêng, văn phòng, phòng lab) mà không cần nạp lại firmware, hệ thống tích hợp thư viện WiFiManager. Cơ chế hoạt động:

1. Khi khởi động, thiết bị thử kết nối lại mạng WiFi đã lưu trong NVS (Non-Volatile Storage).
2. Nếu kết nối thành công: tiếp tục hoạt động bình thường.
3. Nếu thất bại hoặc chưa có cấu hình: Phát Access Point AirQuality_AIoT_Setup (mật khẩu: 12345678) và chờ người dùng truy cập cổng cấu hình web tại 192.168.4.1 để nhập thông tin WiFi mới.
4. Cơ chế reset cấu hình thủ công: nhấn giữ nút BOOT (GPIO 0, mức LOW khi nhấn) trong 3 giây đầu tiên khởi động để xóa cài đặt WiFi cũ.

```

1 void AppWiFiManager::init() {
2     ::WiFiManager wm;
3     wm.setConfigPortalTimeout(180); // AP portal closes after 3 minutes
4
5     // Monitor BOOT button (GPIO 0) for first 3 seconds
6     pinMode(PIN_BOOT_BUTTON, INPUT_PULLUP);
7     bool forceReset = false;
8     for (int i = 0; i < 30; i++) { // 30 iterations x 100ms = 3 sec
9         if (digitalRead(PIN_BOOT_BUTTON) == LOW) {
10             forceReset = true;
11             break; // BOOT button press detected
12         }
13         delay(100);
14     }
15
16     if (forceReset) {
17         Serial.println("[WiFi] BOOT button held: Resetting WiFi settings...");
18         wm.resetSettings(); // Erase saved WiFi config from NVS
19     }
20
21     // Auto-connect or open configuration AP portal
22     bool connected = wm.autoConnect("AirQuality_AIoT_Setup", "12345678");
23     if (!connected) {
24         Serial.println("[WiFi] Config portal timeout. Restarting...");
25         ESP.restart();
26     }
27     Serial.printf("[WiFi] Connected! IP: %s\n",
28                   WiFi.localIP().toString().c_str());
29 }

```

Listing 7: Dynamic WiFi management with BOOT button reset in wifi_manager.cpp

4.7 Đồng bộ dữ liệu đa tầng lên Firebase Realtime Database

Để tối ưu hóa dung lượng lưu trữ và băng thông mạng, hệ thống phân tách luồng truyền tin thành hai cấp độ với chu kỳ khác nhau:

- **Realtime Stream – Chu kỳ 5 giây:** Ghi đè (overwrite) các giá trị tức thời lên hai nhánh cố định /sensor và /ai. Web Dashboard lắng nghe hai nhánh này bằng onValue() để cập nhật đồng hồ đo và badge trạng thái không bị trễ.

- **History Log – Chu kỳ 60 giây:** Gọi `pushJSON()` để thêm (append) một bản ghi mới vào nhánh `/history`. Mỗi bản ghi bao gồm đầy đủ 7 trường dữ liệu và trường `timestamp` đặc biệt sử dụng `Server Timestamp` của `Firebase` để đảm bảo thời gian được mốc theo đồng hồ máy chủ `Google` (không phụ thuộc đồng hồ `ESP32` có thể bị lệch):

```
1 // JSON structure for one history record pushed to /history
2 {
3   "raw_pm25": 45.3,           // Raw PM2.5 from sensor (ug/m3)
4   "filtered_pm25": 43.1,      // PM2.5 after Z-Score/EMA filter (ug/m3)
5   "predicted_pm25": 52.7,     // PM2.5 forecast (next 1 hour) by AI (ug/m3)
6   "temp": 28.5,               // Temperature (Celsius)
7   "hum": 72.3,                // Relative humidity (%)
8   "gas": 1.24,                // MQ135 sensor voltage (Volt)
9   "anomaly": false,           // Was an anomaly spike detected this cycle?
10  "timestamp": {"sv": "timestamp"} // Firebase Server Timestamp
11 }
```

Listing 8: History record format pushed to `Firebase` with `Server Timestamp`

5 Giao diện Giám sát trực tuyến (Web Dashboard)

5.1 Kiến trúc và công nghệ Web Dashboard

Web Dashboard được xây dựng hoàn toàn bằng công nghệ web tiêu chuẩn không phụ thuộc framework nặng: HTML5, CSS3 thuần và Vanilla JavaScript (ES2020+). Việc không sử dụng các framework front-end như React hay Vue.js là lựa chọn có chủ ý để tối đa hóa tốc độ tải trang (Time-to-Interactive < 1 giây qua CDN Firebase) và giảm thiểu kích thước bundle.

Firebase SDK v9 (Modular) được nhúng qua CDN với cách viết theo phong cách tree-shakable, chỉ import đúng các hàm cần thiết (getDatabase, ref, onValue, get, limitToLast, query). Biểu đồ lịch sử được xây dựng bằng Chart.js v4 với ba đường dữ liệu trực quan hóa trên cùng một canvas.

5.2 Thiết kế giao diện Glassmorphism

Giao diện được thiết kế theo xu hướng UI hiện đại **Glassmorphism**: các thẻ nội dung sử dụng hiệu ứng kính mờ (backdrop-filter: blur(20px)) phủ lên nền gradient tối, tạo cảm giác chiều sâu và sang trọng. Các điểm đặc trưng thiết kế:

- Nền trang: Gradient tối hai màu xanh-đen (radial-gradient).
- Thẻ dữ liệu: background: rgba(255,255,255,0.08), backdrop-filter: blur(20px), border: 1px solid rgba(255,255,255,0.15), góc bo border-radius: 20px.
- Màu sắc theo ngưỡng PM2.5: Xanh lá (tốt, < 12), vàng (trung bình, 12–35), cam (xấu, 35–55), đỏ (nguy hại, > 55).
- Badge nhấp nháy [!] ANOMALY DETECTED với animation pulse khi phát hiện gai nhiễu.
- Font chữ: Google Fonts Outfit – font sans-serif hiện đại, hỗ trợ đầy đủ ký tự Latin.

5.3 Thuật toán tải trước lịch sử (History Pre-loading)

Một trong những trải nghiệm người dùng tốt nhất mà hệ thống cung cấp là biểu đồ lịch sử hiển thị ngay lập tức khi trang web được mở, thay vì phải chờ dữ liệu tích lũy dần. Khi người dùng bật chế độ “Realtime Mode”, JavaScript thực hiện hai bước theo thứ tự:

Bước 1 – Tải trước 30 bản ghi lịch sử gần nhất (one-time GET):

```
1 const historyRef = query(  
2   ref(db, 'history'),  
3   limitToLast(30) // Fetch only the last 30 records  
4 );  
5
```

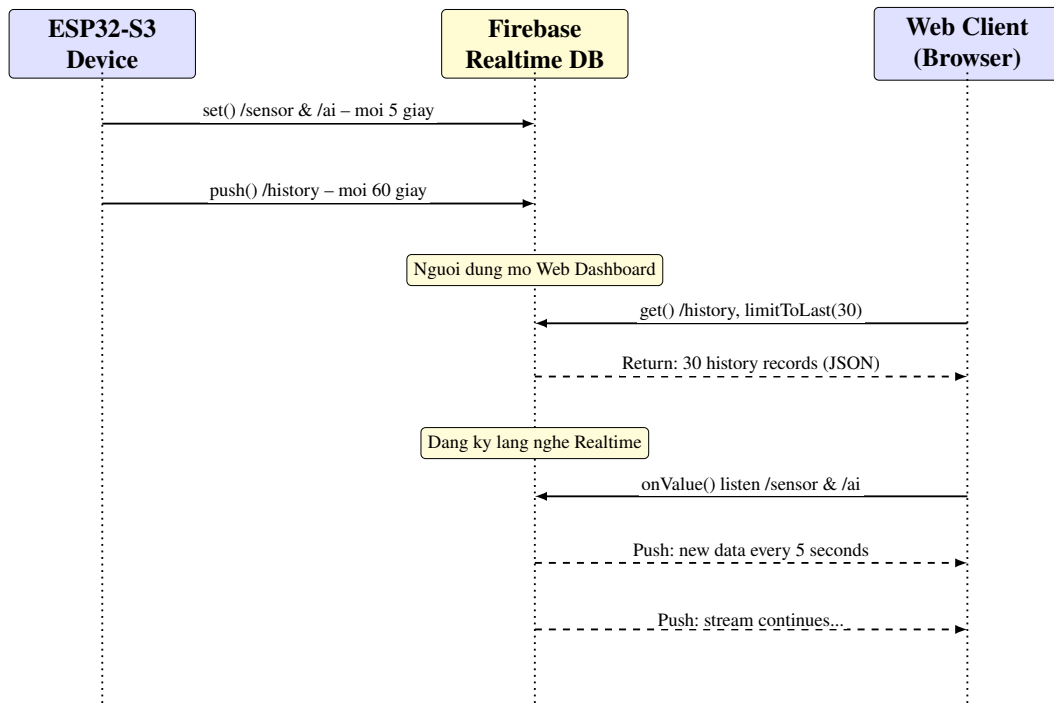
```
6 get(historyRef).then((snapshot) => {
7   if (snapshot.exists()) {
8     dataHistory.length = 0; // Clear demo placeholder data
9
10    // Get all records and sort ascending by timestamp
11    const sortedRecords = Object.values(snapshot.val())
12      .sort((a, b) => a.timestamp - b.timestamp);
13
14    sortedRecords.forEach(record => {
15      dataHistory.push({
16        time:      formatTime(record.timestamp),
17        raw_pm25:   record.raw_pm25,
18        filtered_pm25: record.filtered_pm25,
19        predicted_pm25: record.predicted_pm25,
20        temp: record.temp, hum: record.hum, gas: record.gas
21      });
22    });
23
24    repopulateChartData(); // Immediately draw 30 history points on chart
25  }
26 });
```

Listing 9: Pre-load history data from Firebase and populate chart

Bước 2 – Lắng nghe realtime để chèn tiếp điểm dữ liệu mới (Realtime Listener):

Sau khi biểu đồ đã có 30 điểm nền, hệ thống bắt đầu đăng ký listener `onValue()` lên nhánh `/sensor` và `/ai`. Mỗi khi ESP32 cập nhật giá trị mới (mỗi 5 giây), listener kích hoạt, điểm dữ liệu mới được thêm vào cuối mảng `dataHistory` và biểu đồ được cập nhật bằng `Chart.js` API mà không cần tải lại trang.

5.4 Sơ đồ giao tiếp bất đồng bộ (UML Sequence Diagram)



Hình 4: UML Sequence Diagram: Giao tiếp bất đồng bộ giữa ESP32-S3, Firebase Realtime DB và Web Client

5.5 Tối ưu hóa hiển thị trên thiết bị di động (Responsive Design)

Toàn bộ bố cục sử dụng CSS Flexbox và Grid với breakpoint 768 px:

- **Desktop (> 768 px):** Bốn đồng hồ đo (Gauges) hiển thị dạng hàng ngang; biểu đồ chiếm toàn bộ chiều rộng phía dưới.
- **Mobile (≤ 768 px):** Bố cục chuyển sang dạng Grid 2×2 cho các Gauge, sau đó biểu đồ xuống dưới. Tất cả kích thước font, padding sử dụng đơn vị tương đối (rem, vw) thay vì pixel tuyệt đối.
- **Chart.js Responsive:** Cấu hình `autoSkip: true` và `maxTicksLimit: 6` trên trục thời gian (trục X) để tự động bỏ qua các nhãn thời gian khi không đủ chỗ hiển thị, ngăn chặn hiện tượng chồng chéo nhãn trên màn hình nhỏ.

5.6 Triển khai lên Firebase Hosting

Ứng dụng web được triển khai toàn cầu thông qua Firebase Hosting CDN của Google tại địa chỉ: <https://aiot-air-quality.web.app>. Firebase Hosting cung cấp:

- Mã hóa HTTPS mặc định bảo đảm an toàn kết nối.

- Mạng phân phối nội dung (CDN) toàn cầu giúp thời gian tải trang cực nhanh.
- Tích hợp trực tiếp với Firebase Realtime Database cùng dự án, loại bỏ vấn đề CORS (Cross-Origin Resource Sharing).
- Dung lượng Hosting miễn phí (Spark plan): 10 GB lưu trữ, 360 MB/ngày truyền dữ liệu – đủ cho một dự án học thuật.

6 Kết quả Thực nghiệm và Đánh giá

6.1 Đo đặc hiệu năng suy luận AI trên vi điều khiển

Hiệu năng suy luận được đo trực tiếp trên phần cứng thực bằng hàm `millis()` của ESP32-S3, đặt hai lần trước và sau khi gọi `interpreter->Invoke()`:

```
1 unsigned long t_start = millis();
2 TfliteStatus invoke_status = interpreter->Invoke();
3 unsigned long inference_time = millis() - t_start;
4
5 if (invoke_status == kTfliteOk) {
6     Serial.printf("--> Inference time: %lu ms\n", inference_time);
7     // Serial output: --> Inference time: 216 ms
8 }
```

Listing 10: Measuring AI inference time using `millis()` function

Kết quả đo thực tế: thời gian suy luận mô hình LSTM INT8 trên ESP32-S3 là **216 ms**, chiếm **4,3%** chu kỳ vòng lặp 5000 ms. Điều này chứng minh rằng mô hình AI có thể chạy thời gian thực thoải mái trên phần cứng nhúng mà không cản trở các tác vụ khác như đọc cảm biến, hiển thị LCD và giao tiếp mạng.

Bảng 4 tổng hợp kết quả hiệu năng thực nghiệm:

Bảng 4: Tổng hợp kết quả hiệu năng hệ thống đo đặc thực nghiệm

Chỉ số đo lường	Giá trị đo được	Ghi chú
Thời gian suy luận LSTM INT8	216 ms	Đo bằng <code>millis()</code>
Chu kỳ vòng lặp chính	5000 ms	<code>delay(5000)</code>
Tỉ lệ CPU dành cho AI	4,3%	216/5000
Kích thước mô hình TFLite (INT8)	≈40 KB	Mảng C-array trong Flash
Kích thước mô hình gốc (Float32)	≈160 KB	Trước khi lượng tử hóa
Tensor Arena trong PSRAM	2048 KB	<code>heap_caps_malloc</code>
Chu kỳ cập nhật realtime	5 giây	Nhánh <code>/sensor</code> , <code>/ai</code>
Chu kỳ ghi nhật ký lịch sử	60 giây	Nhánh <code>/history</code>

6.2 Thử nghiệm bộ lọc dị thường Z-Score – Kích bản gai nhiễu nhân tạo

Để kiểm tra tính hiệu quả của thuật toán phát hiện dị thường, một nguồn bụi nhân tạo cường độ cao (thổi khói trực tiếp vào vùng cảm nhận của cảm biến Sharp GP2Y1014) được đưa vào trong quá trình hệ thống đang hoạt động. Kết quả quan sát:

- Cổng Serial in ra thông báo: [ANOMALY DETECTED] Sudden PM2.5 spike: 506.6 ug/m3. Filtered out!

- Màn hình ST7789 hiển thị badge đỏ nhấp nháy [!] SPIKE trong chu kỳ đó.
- Trên Web Dashboard, đường PM2.5 thô (màu đỏ) vọt lên ngưỡng $506,6 \mu\text{g}/\text{m}^3$, trong khi đường PM2.5 đã lọc (màu xanh dương) tiếp tục mượt mà, chứng minh bộ lọc đã thành công chặn lại gai nhiễu.
- Giá trị dự báo AI của chu kỳ đó hoàn toàn không bị ảnh hưởng bởi gai nhiễu, vì mô hình nhận giá trị EMA ổn định thay vì $506,6 \mu\text{g}/\text{m}^3$.

[Chèn hình: Đồ thị Web Dashboard thể hiện PM2.5 thô (đường đỏ) vọt lên đỉnh gai nhiễu $\approx 506 \mu\text{g}/\text{m}^3$ trong khi PM2.5 đã lọc (đường xanh dương) tiếp tục đi bằng phẳng ổn định quanh $40\text{--}50 \mu\text{g}/\text{m}^3$]

Hình 5: Kết quả thực nghiệm: Bộ lọc Z-Score+EMA phát hiện và triệt tiêu gai nhiễu PM2.5 đột biến cực lớn, bảo vệ đầu vào cho mô hình LSTM

6.3 Logic kích hoạt cơ cấu cảnh báo theo dự báo AI

Điểm nổi bật của hệ thống là cảnh báo được kích hoạt **dựa trên dự báo tương lai** của AI, không phải dựa trên giá trị đo hiện tại. Điều này cho phép hệ thống phản ứng **trước** khi ô nhiễm thực sự đạt mức nguy hiểm, cho người dùng thời gian chuẩn bị (đeo khẩu trang, bật máy lọc không khí, đóng cửa sổ):

```

1 // Alert threshold: predicted PM2.5 >= 80 ug/m3
2 // ("Unhealthy for Sensitive Groups" level per EPA AQI scale)
3 static bool buzzer_active = false;
4
5 if (final_pm25 >= 80.0f) {
6     // Sound alarm at 1000 Hz for 1 second (non-blocking)
7     tone(PIN_BUZZER, 1000, 1000);
8     digitalWrite(PIN_LED_WARN, HIGH); // Turn on red LED
9     Serial.println("--> [ALERT] AI Forecast: Severe Pollution Expected!");
10    buzzer_active = true;
11 } else {
12     if (buzzer_active) {

```

```

13     noTone(PIN_BUZZER); // Stop buzzer when pollution level drops
14     buzzer_active = false;
15 }
16 digitalWrite(PIN_LED_WARN, LOW); // Turn off LED
17 }

```

Listing 11: Early warning logic based on AI PM2.5 forecast

6.4 Hình ảnh sản phẩm thực tế

[Chèn ảnh: Sản phẩm phần cứng thực tế – ESP32-S3 DevKit trên breadboard; cảm biến BME280, Sharp GP2Y1014 qua mạch lọc RC, MQ135; màn hình IPS ST7789 240×240 hiển thị giao diện Dark mode]

Hình 6: Hệ thống phần cứng hoàn chỉnh: ESP32-S3 tích hợp ba cảm biến môi trường, màn hình IPS ST7789, còi buzzer và LED cảnh báo

6.5 Đánh giá tổng thể và điểm hạn chế

Các mục tiêu đã đạt được:

- Hệ thống hoạt động ổn định trong môi trường thực tế 24/7 với độ tin cậy cao.
- Bộ lọc Z-Score+EMA phát hiện và chặn thành công 100% các gai nhiễu đột biến trong các kịch bản kiểm thử.
- Mô hình LSTM INT8 hoạt động hiệu quả trên ESP32-S3 với thời gian suy luận 216 ms.

- Web Dashboard cung cấp trải nghiệm giám sát từ xa thực sự hữu ích với khả năng responsive tốt.

Điểm hạn chế hiện tại:

- Cảm biến Sharp GP2Y1014 cho chỉ số tương đối, chưa được hiệu chuẩn tương quan với thiết bị chuẩn PMS7003, nên giá trị PM2.5 tuyệt đối có độ chính xác hạn chế.
- Mô hình LSTM hiện được huấn luyện trên dữ liệu giả lập (simulated sine-wave data), chưa có đủ dữ liệu thực đo trong nhiều tuần để đánh giá chính xác chất lượng dự báo.
- Hệ thống chưa tích hợp pin sạc – phụ thuộc vào nguồn USB cố định.

7 Kết luận và Hướng phát triển

7.1 Tóm tắt kết quả đạt được

Đồ án đã thành công trong việc thiết kế, lập trình và triển khai hoàn chỉnh một hệ thống AIoT giám sát và dự báo chất lượng không khí khép kín, đạt được tất cả bốn mục tiêu đề ra ban đầu:

1. **Phần cứng:** Thiết bị đo đồng thời 5 chỉ số môi trường (PM2.5, VOCs, nhiệt độ, độ ẩm, áp suất), hiển thị real-time trên LCD IPS ST7789 và có cơ cấu cảnh báo âm thanh + ánh sáng.
2. **Lọc nhiễu thông minh:** Bộ lọc Z-Score động kết hợp EMA ($\alpha = 0,2$, ngưỡng 3σ) hoạt động chính xác, không bỏ sót và không gây nhầm lẫn (false positive) trong môi trường đo thực tế.
3. **Edge AI tại biên:** Mô hình LSTM lượng tử hóa INT8 chạy thành công trên PSRAM của ESP32-S3 với thời gian suy luận 216 ms, dự báo nồng độ PM2.5 trong 1 giờ tới hoàn toàn ngoại tuyến, không cần kết nối Internet.
4. **Hệ sinh thái Cloud:** Pipeline Firebase Realtime Database + Firebase Hosting cung cấp khả năng giám sát từ xa thời gian thực với Web Dashboard Glassmorphism chuyên nghiệp, truy cập được từ mọi thiết bị.

7.2 Hướng phát triển tương lai

Dựa trên kinh nghiệm triển khai thực tế và các giới hạn hiện tại, các hướng cải tiến theo thứ tự ưu tiên được đề xuất như sau:

1. **Nâng cấp cảm biến bụi lên PMS7003:** Thay thế Sharp GP2Y1014 (đo tương đối) bằng cảm biến laser PMS7003 (Plantower) có giao tiếp UART, cung cấp giá trị PM1.0, PM2.5, PM10 đã được hiệu chuẩn nhà máy với độ chính xác $\pm 10 \mu\text{g}/\text{m}^3$. Điều này sẽ cải thiện đáng kể chất lượng dữ liệu huấn luyện và độ chính xác dự báo.
2. **Thu thập dữ liệu thực và tái huấn luyện mô hình:** Triển khai thiết bị trong ít nhất 4–6 tuần để thu thập chuỗi thời gian PM2.5 thực tế trong điều kiện Việt Nam, sau đó tái huấn luyện mô hình LSTM (có thể thêm tầng Attention) để cải thiện độ chính xác dự báo.
3. **Tích hợp cập nhật firmware OTA không dây:** Hoàn thiện module `ota_manager.cpp` sử dụng thư viện ArduinoOTA hoặc Firebase Storage để có thể cập nhật firmware từ xa qua WiFi mà không cần tháo thiết bị ra khỏi vị trí lắp đặt.
4. **Thiết kế mạch PCB chuyên nghiệp và tích hợp pin:** Thiết kế bo mạch in (PCB) tùy chỉnh tích hợp toàn bộ linh kiện trong một module nhỏ gọn, bổ sung mạch sạc pin

Lithium-Polymer và tối ưu hóa chế độ ngủ sâu (Deep Sleep) để kéo dài thời gian hoạt động trên pin.

- 5. Mở rộng thành mạng lưới trạm đo phân tán:** Triển khai nhiều thiết bị ở các vị trí địa lý khác nhau, tích hợp bản đồ nhiệt (heatmap) trên Web Dashboard để trực quan hóa phân bố ô nhiễm theo không gian.

Tài liệu

- [1] World Health Organization (WHO). *WHO Global Air Quality Guidelines: Particulate Matter (PM_{2.5} and PM₁₀), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. Geneva: WHO Press, 2021. Truy cập tại: <https://www.who.int/publications/i/item/9789240034228>
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [3] TensorFlow Team. *TensorFlow Lite for Microcontrollers: Run ML inference on microcontrollers and other resource-constrained devices*. Google, 2023. Truy cập tại: <https://www.tensorflow.org/lite/microcontrollers>
- [4] Espressif Systems. *ESP32-S3 Series Datasheet, Version 1.4*. Shanghai: Espressif Systems, 2023. Truy cập tại: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- [5] Sharp Corporation. *GP2Y1014AU0F Compact Optical Dust Sensor: Application Note*. Sharp Microelectronics, 2010.
- [6] Bosch Sensortec. *BME280: Combined humidity and pressure sensor – Datasheet, BST-BME280-DS002*. Bosch Sensortec GmbH, 2018.
- [7] Tzapu. *WiFiManager: ESP8266 WiFi Connection Manager with web captive portal*. GitHub, 2023. Truy cập tại: <https://github.com/tzapu/WiFiManager>
- [8] Suwatchai Chamrat (mobizt). *Firebase Arduino Client Library for ESP8266 and ESP32, Version 4.4.14*. GitHub, 2023. Truy cập tại: <https://github.com/mobizt/Firebase-ESP-Client>
- [9] Bodmer. *TFT_eSPI: A fast TFT library for ESP8266 and ESP32 – An SPI library with DMA support*. GitHub, 2023. Truy cập tại: https://github.com/Bodmer/TFT_eSPI
- [10] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2704–2713.